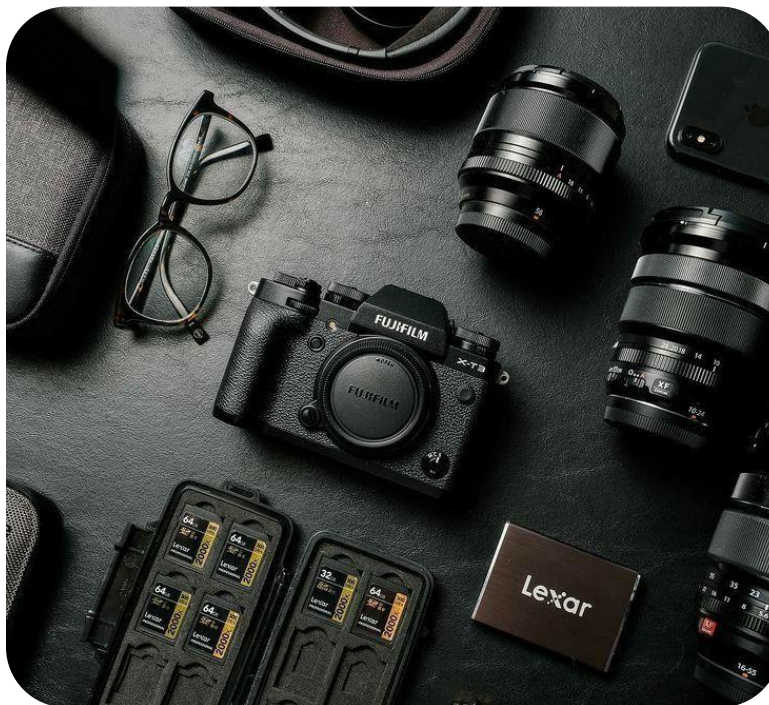




CSC 2032 – Object Oriented Programming (Group Project)

CAMERA EQUIPMENT & INVENTORY HARDWARE SYSTEM



Submitted By:

- AS20240593 – Rushini Keshani
- AS20240626 – A.R Bandara

Table of Contents

Table of Contents.....	2
1. Introduction	3
1.1 Background	3
1.2 Problem Statement.....	3
1.3 Objectives.....	3
1.4 Scope	3
2. System Overview	4
2.1 Proposed Solution	4
2.2 User Roles.....	4
2.3 Use Case Diagram	5
3. Object-Oriented Design.....	5
3.1 Identifying Classes and Responsibilities	6
3.2 Applying Core OOP Principles.....	6
3.2.1 Encapsulation	6
3.2.2 Inheritance	6
3.2.3 Polymorphism	6
3.2.4 Abstraction.....	6
3.3 UML Class Diagram	7
4. Database Design	8
4.1 Entity-Relationship Diagram	8
4.2 Table Summary	8
5. System Architecture	9
5.1 Suggested Package Structure	9
6. Sample Code Snippets	9
6.1 Abstract Superclass — Equipment	10
6.2 Subclass Overriding Behaviour — CameraBody, CameraLens, Accesories	11
6.3 Polymorphism in Action — Inventory Valuation.....	12
6.4 Checkout Business Rule Enforcement.....	14
7. Git Collaboration Workflow.....	15
8. Testing Plan	15
9. Future Enhancements.....	15
10. Conclusion	15

1. Introduction

1.1 Background

Camera bodies, lenses, tripods, lighting rigs, and other production gear are expensive, delicate, and easy to lose track of when a studio or rental business owns more than a handful of items. Equipment frequently moves between storage rooms, shoot locations, and repair shops, and it is borrowed and returned by multiple staff members. Without a structured tracking system, studios commonly face problems such as items going missing, overdue returns, undocumented damage, and no clear record of maintenance history.

This project addresses that real-world problem by designing and developing a Camera Equipment and Inventory Hardware System, a web-based application that allows staff to register equipment, track its physical location, log who has checked an item out and when it is due back, and record maintenance and repair history.

1.2 Problem Statement

Studios and equipment rental services that manage camera bodies, lenses, and accessories need a reliable way to answer three questions at any time:

1. Where is this piece of equipment right now, and who has it?
2. What condition is it in, and when was it last serviced?
3. Is it overdue for return, or due for scheduled maintenance?

Manual logs (spreadsheets or paper sign-out sheets) answer these questions slowly and are prone to human error, especially as the inventory grows. A dedicated OOP-based system models each of these concerns as objects with clear responsibilities, making the logic easier to extend and maintain.

1.3 Objectives

- To design a system using core OOP concepts (Inheritance, Polymorphism, Encapsulation).
- To develop a Java-based backend to manage the equipment logic.
- To create a web interface (HTML/JSP) for users to view the inventory.
- To connect a MySQL database to save equipment details permanently

1.4 Scope

The system covers equipment registration, categorization, location tracking, check-out/check-in workflows, maintenance logging, and basic reporting (e.g. overdue items, items due for service). It does not cover barcode/RFID hardware integration, these are noted as future enhancements in Section 9.

2. System Overview

2.1 Proposed Solution

The system is a three-tier web application:

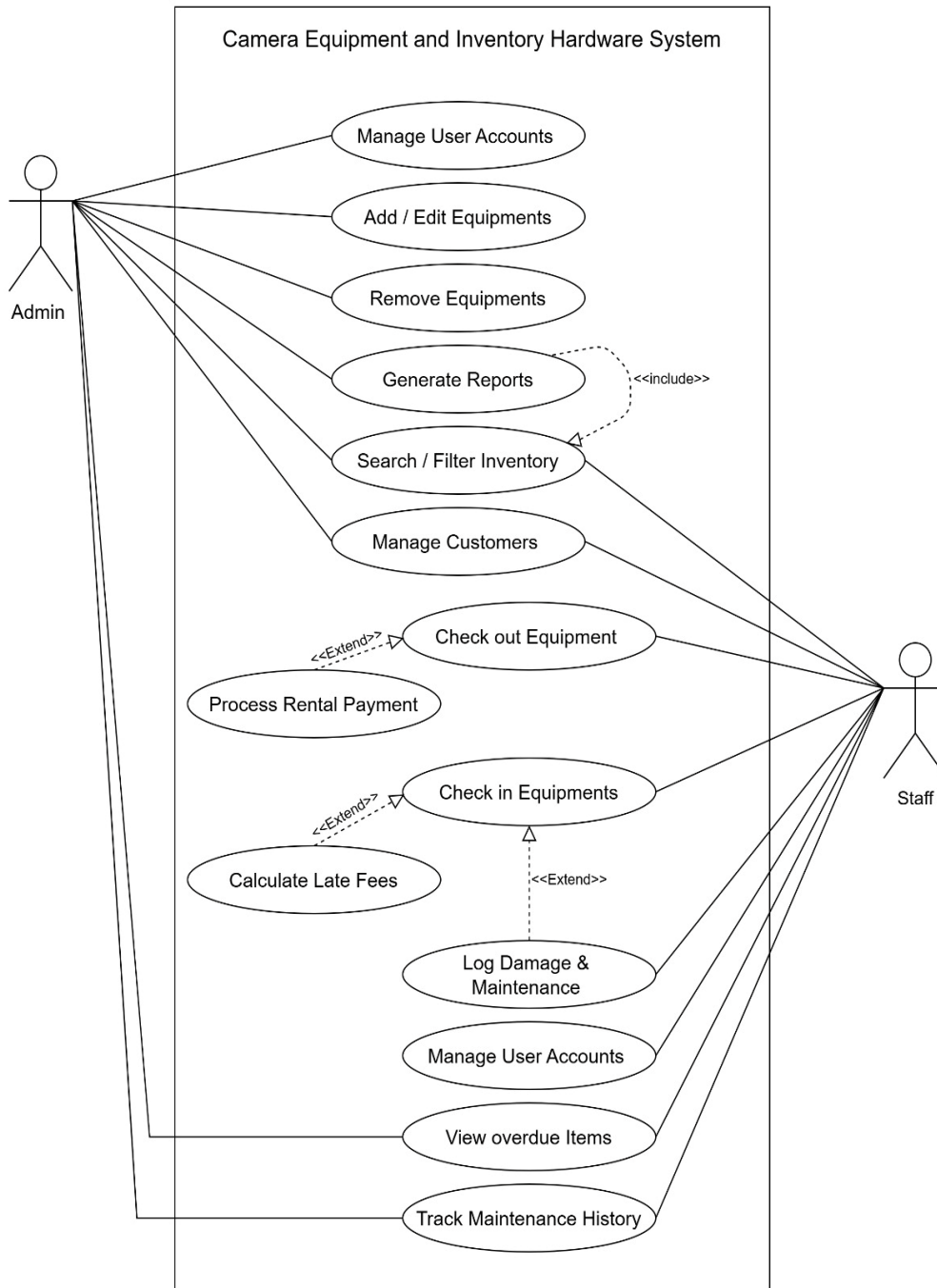
- Front-end: HTML/CSS/JavaScript pages served by JSP, used by Admin and Staff to interact with the system.
- Back-end: Java classes running on Apache Tomcat via Servlets, implementing all business logic using OOP principles.
- Database: MySQL, storing all persistent data (equipment, users, categories, locations, checkout logs, maintenance records).

2.2 User Roles

Role	Description	Key Permissions
Admin	Manages the overall system and oversees inventory health	Add/remove equipment, manage user accounts, view all reports
Staff	Handles day-to-day equipment handling	Check equipment in/out, log maintenance, search inventory

2.3 Use Case Diagram

Figure 1 shows the main interactions between the two user roles and the system.



3. Object-Oriented Design

3.1 Identifying Classes and Responsibilities

The core entities were identified directly from the problem domain: physical equipment, the people who use it, where it is stored, and the events that happen to it (being checked out, or being serviced).

Class	Responsibility
Equipment (abstract)	Common attributes/behaviour shared by all gear (ID, name, serial number, price, condition)
CameraBody, Lens, AccessoryGear	Subclasses adding type-specific attributes and overriding depreciation/maintenance logic
Category	Groups equipment into types (e.g. DSLR, Mirrorless, Prime Lens, Lighting)
MaintenanceRecord	Represents a single service/repair event tied to one equipment item
CheckoutLog	Represents one checkout/check-in transaction
User	Represents an Admin or Staff account

3.2 Applying Core OOP Principles

3.2.1 Encapsulation

Class fields are kept secure by declaring them as private (e.g., in User and MaintenanceRecord). Direct access is restricted, and data is accessed safely using public getters and setters, which are automatically generated using the Lombok @Data annotation.

3.2.2 Inheritance

The system uses a parent-child class hierarchy to prevent code duplication. The Equipment class acts as the base parent class, while specific child classes like CameraBody, CameraLens, and Accessories inherit its common properties (like ID and brand) and add their own unique attributes.

3.2.3 Polymorphism

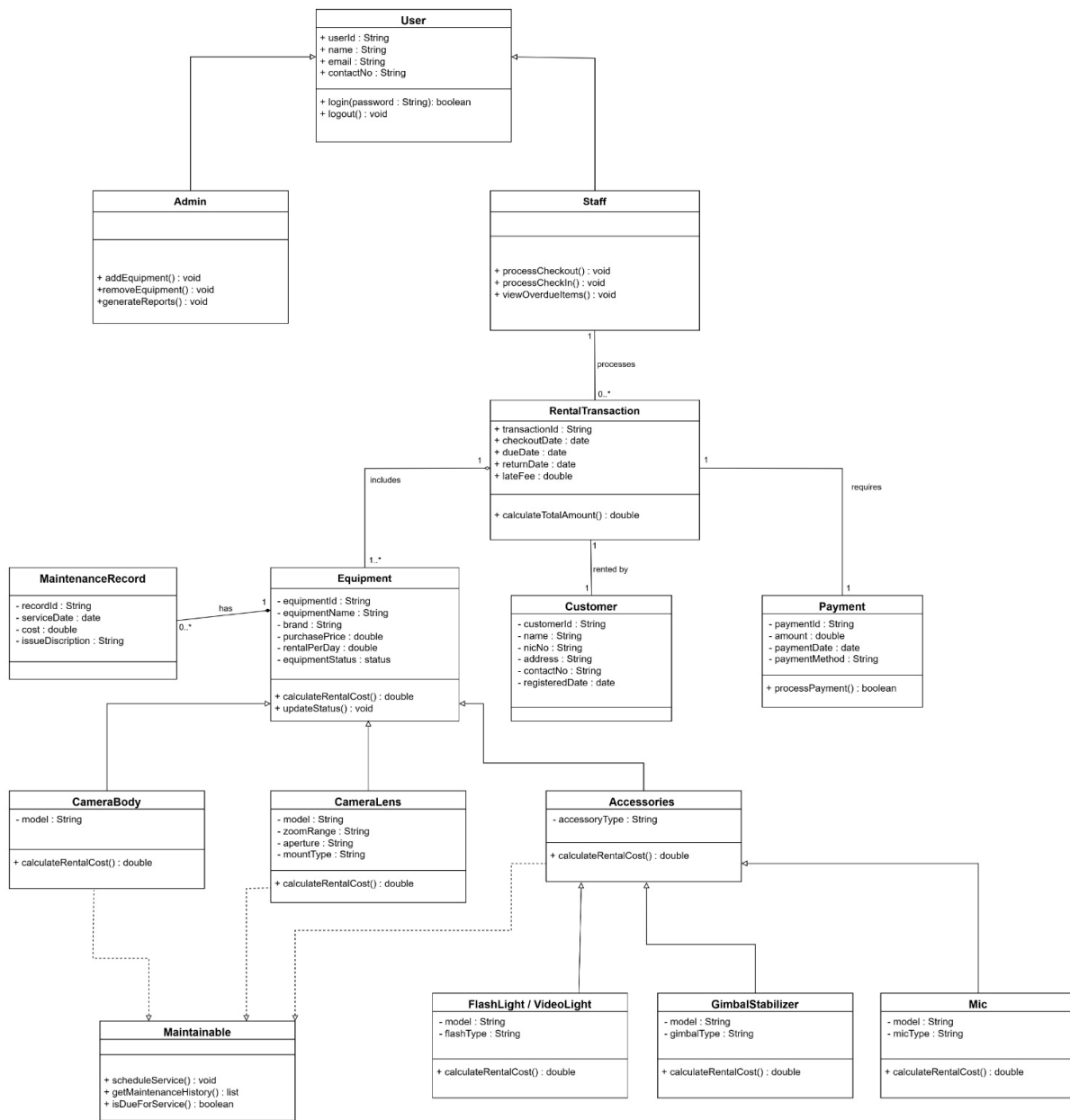
Objects of different subclasses can be treated as objects of a common superclass. For example, in EquipmentServiceImpl, different objects like CameraBody or CameraLens are assigned to a generic Equipment variable and saved using a single equipmentRepository.save(equipment) method.

3.2.4 Abstraction

Complex background logic is hidden from the presentation layer. The system achieves this using interface; the AdminController interacts only with the EquipmentService interface (the "what") and remains completely unaware of the complex database operations handled inside the EquipmentServiceImpl class (the "how").

3.3 UML Class Diagram

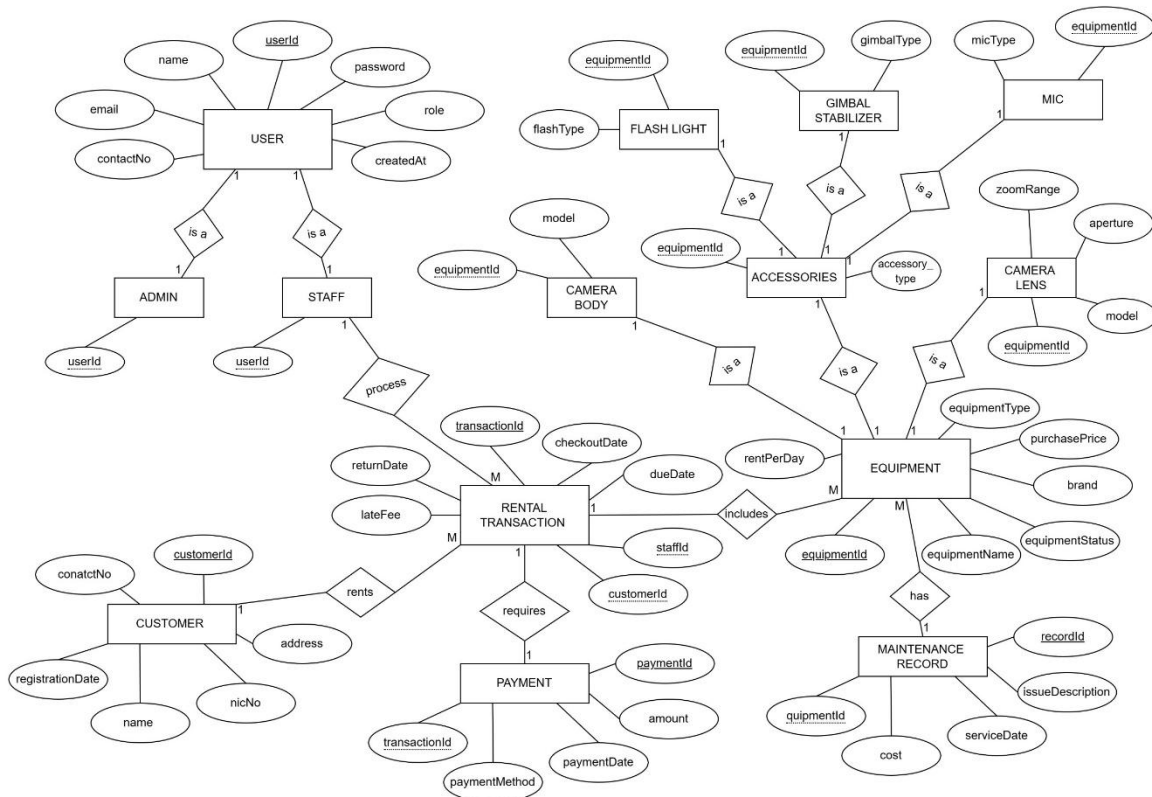
Figure 2 shows the full class structure, including inheritance from Equipment, the two interfaces it implements, and its associations with Category, Location, MaintenanceRecord, CheckoutLog, and User.



4. Database Design

4.1 Entity-Relationship Diagram

Figure 3 shows how the object model maps to relational tables. Note that CameraBody, Lens, and Accessory use table-per-subclass inheritance: each shares a primary key with the parent Equipment table (a one-to-one FK relationship), which mirrors the class inheritance shown in Figure 2.



4.2 Table Summary

Table	Purpose
user	Stores admin and staff accounts and roles
category	Equipment categories (DSLR, Lens, Lighting, etc.)
location	Physical storage locations
equipment	Base table for all equipment (shared attributes)
camera_body / lens / accessory	Subtype tables holding type-specific attributes
maintenance_record	Service and repair history per equipment item

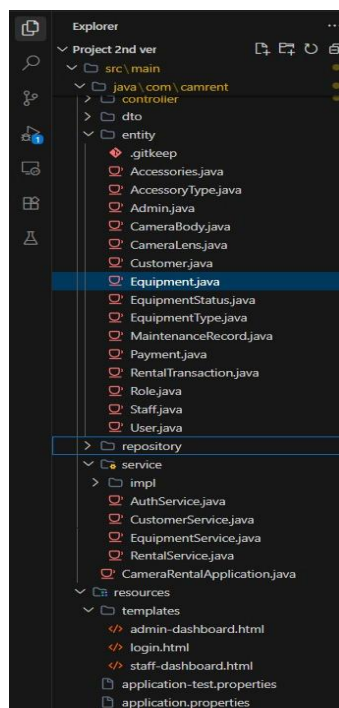
Table	Purpose
checkout_log	Check-out / check-in transaction history

5. System Architecture

The application follows a layered architecture utilizing the Spring Boot framework to ensure maintainability and strict separation of concerns. It uses the Lombok library (@Data annotation) for automatic encapsulation.

- **Controller Layer:** Handles HTTP requests using interface-based abstraction (AdminController).
- **Service Layer:** Contains the core business logic, separating definitions from implementations (EquipmentService and EquipmentServiceImpl).
- **Repository Layer:** Manages database operations, utilizing polymorphism to save different subclasses via a single method (equipmentRepository).
- **Entity Layer:** Contains the database models and OOP class hierarchy (Equipment, CameraBody, CameraLens, Accessories, User, MaintenanceRecord).

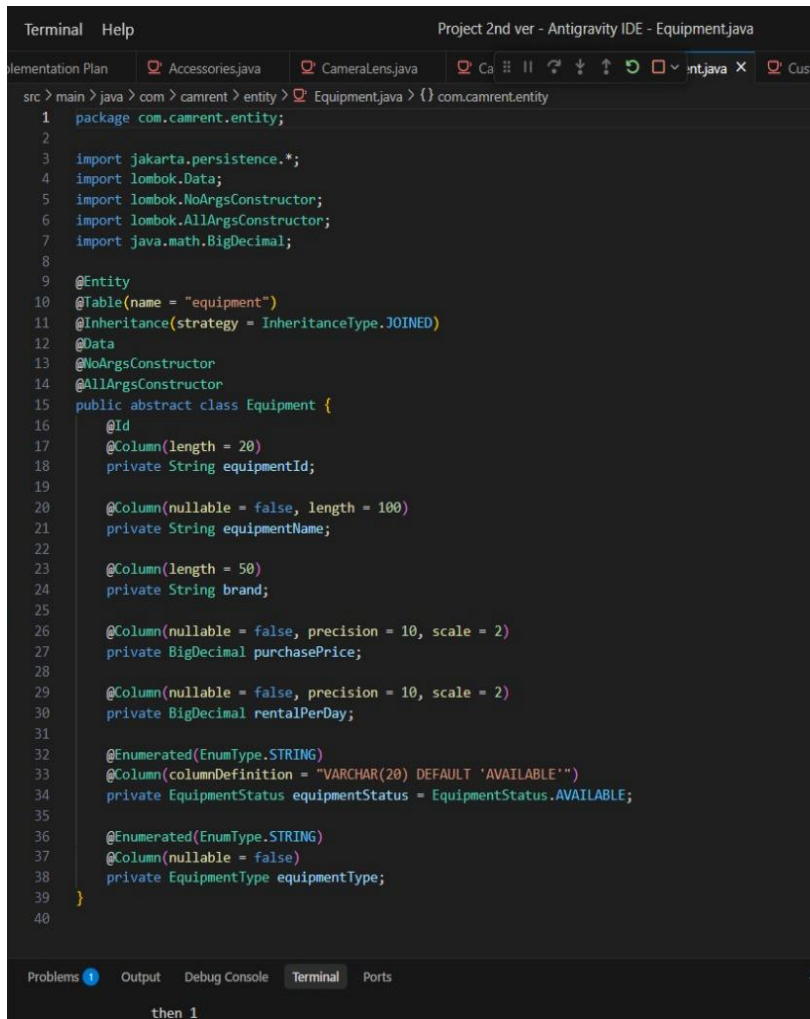
5.1 Suggested Package Structure



6. Sample Code Snippets

The following snippets illustrate how the key OOP principles from Section 3 translate into Java code.

6.1 Abstract Superclass — Equipment



```
Terminal Help Project 2nd ver - AntigraVity IDE - Equipment.java
Implementation Plan Accessories.java CameraLens.java Ca II Equipment.java {} com.camrent.entity
src > main > java > com > camrent > entity > Equipment.java > {} com.camrent.entity
1 package com.camrent.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import lombok.AllArgsConstructor;
7 import java.math.BigDecimal;
8
9 @Entity
10 @Table(name = "equipment")
11 @Inheritance(strategy = InheritanceType.JOINED)
12 @Data
13 @NoArgsConstructor
14 @AllArgsConstructor
15 public abstract class Equipment {
16     @Id
17     @Column(length = 20)
18     private String equipmentId;
19
20     @Column(nullable = false, length = 100)
21     private String equipmentName;
22
23     @Column(length = 50)
24     private String brand;
25
26     @Column(nullable = false, precision = 10, scale = 2)
27     private BigDecimal purchasePrice;
28
29     @Column(nullable = false, precision = 10, scale = 2)
30     private BigDecimal rentalPerDay;
31
32     @Enumerated(EnumType.STRING)
33     @Column(columnDefinition = "VARCHAR(20) DEFAULT 'AVAILABLE'")
34     private EquipmentStatus equipmentStatus = EquipmentStatus.AVAILABLE;
35
36     @Enumerated(EnumType.STRING)
37     @Column(nullable = false)
38     private EquipmentType equipmentType;
39 }
40
```

Problems 1 Output Debug Console Terminal Ports

then 1

6.2 Subclass Overriding Behaviour

- CameraLens

```
src > main > java > com > camrent > entity > CameraLens.java > {
2
3  import jakarta.persistence.Column;
4  import jakarta.persistence.Entity;
5  import jakarta.persistence.Table;
6  import lombok.Data;
7  import lombok.EqualsAndHashCode;
8  import lombok.NoArgsConstructor;
9  import lombok.AllArgsConstructor;
10
11  @Entity
12  @Table(name = "camera_lens")
13  @Data
14  @EqualsAndHashCode(callSuper = true)
15  @NoArgsConstructor
16  @AllArgsConstructor
17  public class CameraLens extends Equipment {
18      @Column(length = 100)
19      private String model;
20
21      @Column(length = 50)
22      private String zoomRange;
23
24      @Column(length = 20)
25      private String aperture;
26
27      @Column(length = 50)
28      private String mountType;
29  }
```

- Accessories

```
src > main > java > com > camrent > entity > Accessories.java > {} com.camrent.en
1  package com.camrent.entity;
2
3  import jakarta.persistence.Column;
4  import jakarta.persistence.Entity;
5  import jakarta.persistence.EnumType;
6  import jakarta.persistence.Enumerated;
7  import jakarta.persistence.Table;
8  import lombok.Data;
9  import lombok.EqualsAndHashCode;
10 import lombok.NoArgsConstructor;
11 import lombok.AllArgsConstructor;
12
13 @Entity
14 @Table(name = "accessories")
15 @Data
16 @EqualsAndHashCode(callSuper = true)
17 @NoArgsConstructor
18 @AllArgsConstructor
19 public class Accessories extends Equipment {
20     @Enumerated(EnumType.STRING)
21     @Column(nullable = false, source = camrent)
22     private AccessoryType accessoryType;
23 }
24
```

- CameraBody

```
src > main > java > com > camrent > entity > CameraBody.java
1  package com.camrent.entity;
2
3  import jakarta.persistence.Column;
4  import jakarta.persistence.Entity;
5  import jakarta.persistence.Table;
6  import lombok.Data;
7  import lombok.EqualsAndHashCode;
8  import lombok.NoArgsConstructor;
9  import lombok.AllArgsConstructor;
10
11  @Entity
12  @Table(name = "camera_body")
13  @Data
14  @EqualsAndHashCode(callSuper = true)
15  @NoArgsConstructor
16  @AllArgsConstructor
17  public class CameraBody extends Equipment {
18      @Column(length = 100)
19      private String model;
20  }
```

6.4 Encapsulation

```
src > main > java > com > camrent > entity > User.java > {} com.camrent.entity
1 package com.camrent.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import lombok.AllArgsConstructor;
7 import java.time.LocalDateTime;
8
9 @Entity
10 @Table(name = "users")
11 @Inheritance(strategy = InheritanceType.JOINED)
12 @Data
13 @NoArgsConstructor
14 @AllArgsConstructor
15 public class User {
16
17     @Id
18     @Column(length = 20)
19     private String userId;
20
21     @Column(nullable = false, length = 100)
22     private String name;
23
24     @Column(nullable = false, length = 100, unique = true)
25     private String email;
26
27     @Column(length = 20)
28     private String contactNo;
29
30     @Column(nullable = false)
31     private String password;
32
33     @Enumerated(EnumType.STRING)
34     @Column(nullable = false)
35     private Role role;
36
37     @Column(updatable = false)
38     private LocalDateTime createdAt;
39
40     @PrePersist
41     protected void onCreate() {
42         if (this.createdAt == null) {
43             this.createdAt = LocalDateTime.now();
44         }
45     }
46 }
```

```
src > main > java > com > camrent > entity > User.java > {} com.camrent.entity
1 package com.camrent.entity;
2
3 import jakarta.persistence.*;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import lombok.AllArgsConstructor;
7 import java.time.LocalDateTime;
8
9 @Entity
10 @Table(name = "users")
11 @Inheritance(strategy = InheritanceType.JOINED)
12 @Data
13 @NoArgsConstructor
14 @AllArgsConstructor
15 public class User {
16
17     @Id
18     @Column(length = 20)
19     private String userId;
20
21     @Column(nullable = false, length = 100)
22     private String name;
23
24     @Column(nullable = false, length = 100, unique = true)
25     private String email;
26
27     @Column(length = 20)
28     private String contactNo;
29
30     @Column(nullable = false)
31     private String password;
32
33     @Enumerated(EnumType.STRING)
34     @Column(nullable = false)
35     private Role role;
36
37     @Column(updatable = false)
38     private LocalDateTime createdAt;
39
40     @PrePersist
41     protected void onCreate() {
42         if (this.createdAt == null) {
43             this.createdAt = LocalDateTime.now();
44         }
45     }
46 }
```

6.4 Polymorphism

```
src > main > java > com > camrent > service > impl > EquipmentServiceImpl.java > {} com.camrent.service.impl
1 package com.camrent.service.impl;
2
3 import com.camrent.entity.*;
4 import com.camrent.dto.EquipmentDto;
5 import com.camrent.repository.EquipmentRepository;
6 import com.camrent.service.EquipmentService;
7 import lombok.RequiredArgsConstructor;
8 import org.springframework.stereotype.Service;
9 import java.math.BigDecimal;
10 import java.util.List;
11
12 @Service
13 @RequiredArgsConstructor
14 public class EquipmentServiceImpl implements EquipmentService {
15
16     private final EquipmentRepository equipmentRepository;
17
18     @Override
19     public Equipment addEquipment(EquipmentDto dto) {
20         Equipment equipment;
21         switch (dto.getEquipmentType()) {
22             case CAMERA_BODY:
23                 CameraBody body = new CameraBody();
24                 body.setModel(dto.getModel());
25                 equipment = body;
26                 break;
27             case CAMERA_LENS:
28                 CameraLens lens = new CameraLens();
29                 lens.setZoomRange(dto.getZoomRange());
30                 lens.setMountType(dto.getMountType());
31                 equipment = lens;
32                 break;
33             case ACCESSORY:
34                 Accessories acc = new Accessories();
35                 acc.setAccessoryType(dto.getAccessoryType());
36                 equipment = acc;
37                 break;
38             default:
39                 throw new IllegalArgumentException("Unknown equipment type");
40         }
41     }
42 }
```

```

41     equipment.setEquipmentId(dto.getEquipmentId());
42     equipment.setEquipmentName(dto.getEquipmentName());
43     equipment.setBrand(dto.getBrand());
44     equipment.setPurchasePrice(dto.getPurchasePrice());
45     equipment.setRentalPerDay(dto.getRentalPerDay());
46     equipment.setEquipmentStatus(EquipmentStatus.AVAILABLE);
47     equipment.setEquipmentType(dto.getEquipmentType());
48
49     return equipmentRepository.save(equipment);
50 }
51
52 @Override
53 public void removeEquipment(String equipmentId) {
54     equipmentRepository.deleteById(equipmentId);
55 }
56
57 @Override
58 public Equipment updateStatus(String equipmentId, EquipmentStatus status) {
59     Equipment equipment = equipmentRepository.findById(equipmentId).orElseThrow();
60     equipment.setEquipmentStatus(status);
61     return equipmentRepository.save(equipment);
62 }
63
64 @Override
65 public Equipment updateRentalPrice(String equipmentId, BigDecimal newPrice) {
66     Equipment equipment = equipmentRepository.findById(equipmentId).orElseThrow();
67     equipment.setRentalPerDay(newPrice);
68     return equipmentRepository.save(equipment);
69 }
70
71 @Override
72 public List<Equipment> getAllEquipment() {
73     return equipmentRepository.findAll();
74 }
75
76 @Override
77 public List<Equipment> getEquipmentByType(EquipmentType type) {
78     return equipmentRepository.findAll().stream()
79         .filter(e -> e.getEquipmentType() == type)
80         .toList();
81 }
82
83 @Override
84 public List<Equipment> getAvailableEquipmentByType(EquipmentType type) {
85     return equipmentRepository.findByEquipmentTypeAndEquipmentStatus(type, EquipmentStatus.AVAILABLE);
86 }
87 }
88

```


7. Git Collaboration Workflow

- Create a shared GitHub repository with main and develop branches.
- Each member works on feature branches (e.g. feature/checkout-module) and opens pull requests for review.
- Use commit messages that reference the class/feature being worked on, to make the contribution history easy to assess.

8. Testing Plan

Test Case	Input	Expected Result
Add equipment with negative price	purchasePrice = -500	IllegalArgumentException thrown; item not saved
Check out an already checked-out item	Item currently checked out by another staff member	IllegalStateException thrown; checkout rejected
Check in overdue item	Item with dueDate in the past	Item marked returned; flagged as 'returned late'
Search inventory by category	category = 'Lens'	Only lens-type equipment returned
Calculate depreciation for each subtype	List of CameraBody, Lens, AccessoryGear	Each uses its own overridden formula (polymorphism)

9. Future Enhancements

- Barcode or QR-code scanning for faster check-in/check-out.
- Email/SMS notifications for overdue items or upcoming scheduled maintenance.
- Customer-facing booking portal if the system expands beyond internal staff use.
- Analytics dashboard showing utilization rate per equipment item.

10. Conclusion

This project successfully uses Object-Oriented Programming to solve a real-world problem: safely tracking high-value camera gear. By modeling different equipment types through inheritance, encapsulation, abstraction, and polymorphism, we built a secure and practical backend. Combined with a working web interface and a database, this system provides a complete, functional solution that fully meets all the requirements of the CSC 2032 module.

Thank You!